HAUTE COUTURE DIGITAL ATELIER & APPAREL PLATFORM

NOIR & GOLD

Complete Technical Documentation & Engineering Handbook

System Architecture, Full-Stack Design, API Reference, Database Modeling, Security, and Learning Roadmap

ENGINEERING SPECIFICATIONS & AUDIT SUMMARY

Platform Architecture	**Next.js 16 (App Router) + Express 4 Monorepo**
Frontend & UI Engine	**React 19.3.0 + Tailwind CSS v4 + Turbopack**
Database & Persistence	**SQLite 3 + Prisma ORM 5.22 (14 Relational Models)**
Security & Governance	**HMAC-SHA256 Webhooks, RBAC, Dual Rate Limiters**
Testing & Quality Gates	**122 Passing Automated Integration Tests (100% Green)**
Code Quality Status	**0 ESLint Errors/Warnings, Strict TypeScript Typecheck**
Technical Documentation	**14 Comprehensive Educational Modules in docs/**

AUTHOR & SYSTEMS ARCHITECT

Principal Full-Stack Engineer & Application Security Auditor

Publication Date: October 2026 | Document Ref: NG-ARCH-2026-FINAL | Enterprise Edition

TABLE OF CONTENTS

This engineering handbook compiles 14 exhaustive technical modules, architectural diagrams, data models, and API specifications for the NOIR & GOLD luxury e-commerce platform.

1. Project Overview & Executive Summary p. 3

2. Technology Stack & Tooling Guide p. 12

3. Repository Architecture & File Map p. 16

4. Frontend Architecture & App Router Deep-Dive p. 19

5. Backend Architecture & Express REST API p. 27

6. Complete API Reference p. 30

7. Database Design & Prisma ORM Deep-Dive p. 37

8. Authentication, Authorization & Security Architecture p. 47

9. E-Commerce Business Workflows & State Machines p. 54

10. Developer Environment Setup & CLI Reference p. 57

11. Testing Strategy & Test Suite Guide p. 60

12. Production Deployment & Operations Guide p. 63

13. Full-Stack Engineer Learning Roadmap p. 66

14. Technical Glossary & Senior Interview Guide p. 70

1. PROJECT OVERVIEW & EXECUTIVE SUMMARY

1. Executive Summary & Brand Identity

NOIR & GOLD is an haute couture digital atelier and luxury e-commerce platform designed for discerning clientele seeking artisanal Pakistani garments, bespoke silhouettes, and premium fabrics. Combining minimalist, editorial aesthetics (deep obsidian #0D0D0D, rich champagne gold #C9A84C, warm parchment #F9F6F0) with modern full-stack web engineering, the platform delivers an immersive shopping experience and a comprehensive administrative command center.

The application is structured as a TypeScript monorepo powered by:

- Frontend Presentation & APIs: Next.js 16 (App Router), React 19, Tailwind CSS v4, and Prisma ORM client.
- Dedicated Backend Service: Express 4 REST API, Prisma ORM, CORS, and Morgan logging.
- Relational Data Persistence: SQLite database (backend/prisma/dev.db) encompassing 14 synchronized tables with atomic transactions and soft-delete capabilities.
- Payment & Financial Architecture: Support for Cash on Delivery (COD), local Pakistani electronic gateways (JazzCash, EasyPaisa, direct bank transfer), HMAC-SHA256 authenticated webhooks, capacity-checked refund issuance, and RFC-4180 streaming financial CSV exports.

2. Core Problem Solved & Business Context

Traditional luxury fashion platforms often struggle with three fundamental challenges:

1. Inventory Concurrency & Overselling: High-demand drop models risk overcommitting scarce stock when multiple customers attempt simultaneous checkouts. NOIR & GOLD resolves this through atomic inventory holds, optimistic locking, and state-machine-driven reservations.
2. Administrative Friction: Artisanal luxury collections require dynamic variant creation (sizes, bespoke colorways, custom SKUs), rich high-resolution photography pipelines, and fine-grained order status tracking. The atelier provides custom-built admin tooling with live variant matrices, drag-and-drop multipart upload pipelines, and real-time financial auditing.
3. Regional Checkout Adaptability: Serving both domestic Pakistani and international markets demands flexible payment handling, multi-zone shipping rate calculation (Sindh, Punjab, Islamabad/KPK/Balochistan), and strict fraud/rate-limiting defenses.

3. Live Functional Inventory & Feature Completion Matrix

Domain	Feature	Engineering Implementation	Operational Status
Catalog	Public Garment Listing	Server/Client components, category filtering, search, pagination	' Live & Operational
Catalog	Garment Detail View	Dynamic gallery, interactive size/color picker, stock checks	' Live & Operational
Cart	Cart Management	Client-side reactive persistence, stock threshold validation	' Live & Operational
Checkout	Multi-Zone Delivery	Dynamic calculation based on regional shipping zones & free delivery thresholds	' Live & Operational
Checkout	Payment Selection	COD, JazzCash, EasyPaisa, Bank Transfer with verification tokens	' Live & Operational
Orders	Public Order Tracking	Tokenized HMAC guest access (/orders/[id]), state machine timeline	' Live & Operational
Admin	Role-Based Access Control	Cookie sessions, HMAC origin/CSRF checks, ADMIN vs STAFF segregation	' Live & Operational
Admin	Catalog CRUD	Create/Edit garments, soft-delete	

archive & restore

' Live & Operational

Admin

Variant Matrix Builder

Dynamic SKU generation, color hex mapping, inventory allocation

' Live & Operational

Domain	Feature	Engineering Implementation	Operational Status
Admin	Image Upload Pipeline	Multipart form handler, 5 MB payload limit, local & S3 storage abstraction	' Live & Operational
Admin	Financial Ledger & Orders	Filterable order management, customer delivery inspection, payment logs	' Live & Operational
Admin	Refund Processing	Atomic capacity checks, cumulative balance enforcement, event audit trail	' Live & Operational
Admin	CSV Data Export	Streaming RFC-4180 export with complex date/status filtering	' Live & Operational
Security	Rate Limiting	Dual checkout rate limiter (phone and IP abuse prevention)	' Live & Operational
Security	Webhook Verification	HMAC-SHA256 signature enforcement, 300-second timestamp freshness window	' Live & Operational

4. End-to-End User Journeys

4.1 Customer Journey: Discover to Delivery

```
flowchart TD
    A[Visitor Enters Homepage] --> B[Browse Curated Collection]
    B --> C[Select Garment & View Details]
    C --> D[Select Size, Color & Quantity]
    D --> E[Add to Shopping Bag]
    E --> F[Proceed to Checkout]
    F --> G[Provide Delivery Address & Zone]
    G --> H[Select Payment Method: COD / Gateway]
    H --> I[Submit Order & Hold Reservation]
    I --> J[Order Confirmation & HMAC Token Issued]
    J --> K[Real-Time Tracking & Timeline]
```

1. Discovery & Exploration: The customer browses the haute couture collection filtered by category (Bridal, Formal, Pret, Menswear) or search term.
2. Product Engagement: On the product detail page, high-resolution imagery displays fabric textures. Variant selectors dynamically update available stock, price deltas, and SKU identification.
3. Shopping Bag: Items are staged in a persistent shopping bag. Items exceeding remaining physical stock trigger immediate feedback.
4. Checkout & Zone Calculation: Customer enters recipient details. Shipping charges are evaluated against destination zones (e.g., Sindh vs. KPK) and waived if cart total surpasses the free-shipping threshold (e.g., PKR 15,000).
5. Order Finalization: Upon submission, an atomic transaction decreases variant stock, creates an Order record with order items, records an InventoryReservation, and issues a cryptographically secure guest lookup token.
6. Guest Tracking: The customer is redirected to /orders/[id]?token=... displaying live delivery status, courier tracking (TCS, Leopards, DHL), and payment receipt.

4.2 Administrative Journey: Catalog, Fulfillment & Governance

```
flowchart TD
    A[Admin Login with Secret Key] --> B[Session Authenticated: ADMIN or STAFF]
    B --> C[Atelier Dashboard Analytics]
    C --> D[Product Manager: Create / Edit / Archive]
    C --> E[Upload Product Media Assets]
    C --> F[Order Manager: Update Status & Courier]
    C --> G[Process Full / Partial Refunds]
    C --> H[Stream Financial CSV Exports]
```

1. Secure Access: Administrators authenticate using email, password, and the administrative passphrase (ADMIN_SESSION_SECRET). Sessions are signed and stored in HTTP-only, SameSite cookies.
2. Catalog Management: The administrator creates new silhouettes or updates existing garments. The VariantMatrixBuilder generates SKUs across colorways and dimensions. Soft-delete functionality permits archiving garments without breaking historical customer orders.
3. Asset Ingestion: Media is uploaded via drag-and-drop. Assets are validated for MIME type, renamed with collision-resistant UUIDs, and written to persistent storage (/public/uploads or S3).
4. Fulfillment Execution: In the Orders Manager, staff update order progressions (PENDING -> PROCESSING -> SHIPPED -> DELIVERED). Assigning a courier and tracking code automatically updates customer tracking views.
5. Ledger & Refunds: For cancellations or returns, admins trigger atomic refunds against captured payments. The system strictly prevents refunds exceeding the original captured sum and records every event in SecurityAuditLog.
6. Data Intelligence: Administrators export filtered transaction records as RFC-4180 CSV files directly to their local workstation for accounting and inventory audits.

5. System Architecture Overview

```
flowchart LR
    subgraph Client Tier
        Browser[Modern Web Browser / Mobile Device]
    end

    subgraph Edge & Frontend Next.js 16
        AppRouter[Next.js App Router /pages & /api]
        AuthGuard[Admin Auth Guard & RBAC]
        UploadService[Storage Adapter]
        PrismaFrontend[Prisma ORM Client]
    end

    subgraph Backend Services Express
        ExpressApp[Express REST API :5000]
        Controllers[Category & Product Controllers]
        PrismaBackend[Prisma ORM Client]
    end

    subgraph Persistence Layer
        SQLite[(SQLite Database dev.db)]
        UploadStorage[(Local Disk / S3 Assets)]
    end

    Browser <-->|HTTP / HTML / JSON| AppRouter
    Browser <-->|REST API /products| ExpressApp
    AppRouter --> AuthGuard
    AppRouter --> UploadService --> UploadStorage
    AppRouter --> PrismaFrontend --> SQLite
    ExpressApp --> Controllers --> PrismaBackend --> SQLite
```

- Monorepo Cohesion: Next.js 16 App Router handles server-rendered storefront views, customer mutations, authenticated admin dashboards, and specialized edge-like API routes. Express 4 operates as a high-throughput REST service for catalog ingestion, microservice decoupling, and external integration points.
- Shared Relational Foundation: Both frontend and backend connect to the common SQLite database (backend/prisma/dev.db), utilizing generated Prisma client definitions for type-safe relational queries.

6. Prerequisites & Quickstart Overview

To run NOIR & GOLD locally, your workstation requires:

- Node.js: v20.17.0+ (LTS recommended)
- Package Manager: npm (Workspaces enabled)
- Operating System: Windows, macOS, or Linux
- Environment Configuration: Configured .env files in both frontend/ and backend/.

Quickstart Execution:

```
# 1. Clone repository & install dependencies
git clone <repository_url>
npm install

# 2. Synchronize database schema and seed data
npm run backend:db:push
npm run backend:db:seed

# 3. Launch full stack (Concurrent development servers)
# Terminal 1: Launch Backend API (Port 5000)
npm run backend:dev

# Terminal 2: Launch Frontend Atelier (Port 3000)
npm run frontend:dev
```

7. Known Limitations & Planned Enhancements

Feature Area	Current Implementation	Planned Enhancement
Database	SQLite single-file relational database (dev.db)	Migration to managed PostgreSQL (e.g., Supabase / Neon) for high concurrency
Search	Case-insensitive SQL contains queries	Full-text indexing via PostgreSQL tsvector or Algolia search index
Media Storage	Local filesystem with S3 credential fallback	Multi-region CDN distribution via Cloudflare Images or AWS CloudFront
Authentication	Encrypted session cookie with secret passphrase	NextAuth / Auth.js with OAuth 2.0 (Google, Apple) and WebAuthn biometrics
Internationalization	PKR pricing and English language interface	Multi-currency (USD, EUR, AED, GBP) with real-time exchange rates and Urdu localization

2. TECHNOLOGY STACK & TOOLING GUIDE

1. Complete Technology Inventory

The NOIR & GOLD repository employs an enterprise TypeScript stack engineered for type safety, sub-second latency, and resilience. Below is the audited inventory of every core dependency, framework, and tool:

Package / Tool	Version	Architecture Role	Problem Solved
Next.js	16.4.0	Frontend Framework & SSR	App Router file-based routing, Server Components, streaming SSR, API routes
React	19.3.0	UI Component Engine	Declarative component hierarchy, modern hooks, fine-grained DOM reconciliation
React DOM	19.3.0	Browser DOM Renderer	Client rendering, hydration, and event handling
TypeScript	^5.6.3 / ^5	Programming Language	Compile-time static analysis, structural typing, strict null safety
Prisma ORM	^5.22.0	Database ORM & Migrations	Type-safe query generation, SQLite synchronization, relational modeling
SQLite	3.x	Relational Storage Engine	Zero-configuration, file-based relational database ideal for development
Tailwind CSS	^4.0.0	Styling & Design System	Utility-first CSS, high-performance Turbopack build engine, design tokens
Express	^4.21.2	Backend REST Microservice	High-performance routing, microservice endpoints, external service proxy
Morgan	^1.12.1	HTTP Request Logger	Express access logging, timing metrics, and development debugging
CORS	^2.8.5	Cross-Origin Middleware	Cross-origin resource sharing policy between frontend and backend
AWS SDK S3	^3.1152.0	Cloud Object Storage	Multipart S3/Cloudflare R2 image asset uploading
clsx & tailwind-merge	^2.1.1 & ^3.7.0	Class Name Composition	Conditional styling without CSS class collision
tsx	^4.19.2	TypeScript Execution Engine	Zero-compilation TypeScript execution for tests, seeding, and server watch
ESLint	^9.0.0	Static Code Analysis	Linting, React 19 hook safety checks, dead code identification
PDFKit	^0.15.0	PDF Compilation Engine	Programmatic document synthesis, vector typography, and report generation

2. Why Each Technology Was Chosen & Alternative Comparisons

2.1 Next.js 16 (App Router) vs. Traditional SPA (Vite / CRA)

- Why Chosen: Luxury e-commerce demands instant First Contentful Paint (FCP) and optimal SEO for garment discovery. Next.js 16 App Router renders initial markup on the server, avoiding blank screen flashes while streaming hydrated components to the browser.
- Alternatives Considered: Vite + React SPA. Rejected because single-page applications defer all rendering to client JavaScript, degrading SEO for search crawlers and increasing initial bundle load times on mobile devices.

2.2 React 19 vs. React 18

- Why Chosen: React 19 introduces modernized hook validation, optimized server actions, enhanced asset preloading, and

cleaner hydration diagnostics.

- Design Impact: Strict enforcement of lifecycle purity eliminates race conditions during asynchronous catalog fetching.

2.3 Prisma ORM vs. Raw SQL / TypeORM

- Why Chosen: Prisma auto-generates TypeScript types directly from the schema file (schema.prisma). Any change in the data model (such as adding deletedAt soft deletes) immediately propagates type definitions across the entire codebase.
- Alternatives Considered: TypeORM / Knex. Rejected due to manual type synchronization, verbose migration syntax, and lack of compile-time relation verification.

2.4 Tailwind CSS v4 vs. CSS Modules / Styled Components

- Why Chosen: Tailwind CSS v4 integrates natively with Turbopack, providing near-instant hot-module replacement and stripping unused classes to produce production stylesheets smaller than 25 KB.
- Alternatives Considered: Styled Components. Rejected because CSS-in-JS incurs runtime JavaScript evaluation overhead that hampers client performance.

---

3. Concrete Code Implementations in NOIR & GOLD

3.1 Type-Safe Prisma Queries with Soft-Delete Filtering

```
// frontend/src/lib/catalog-service.ts
import { prisma } from "@/lib/prisma";

export async function getActiveGarments(categorySlug?: string) {
  return await prisma.product.findMany({
    where: {
      deletedAt: null, // Standardized soft-delete filter
      isSoldOut: false,
      ...(categorySlug ? { category: { slug: categorySlug } } : {}),
    },
    include: {
      variants: {
        where: { isActive: true },
        select: { id: true, size: true, colorName: true, stock: true, sku: true, pricePkr: true
      },
      images: {
        orderBy: { displayOrder: "asc" },
      },
    },
    orderBy: { createdAt: "desc" },
  });
}
```

3.2 Dynamic Class Name Composition with clsx & twMerge

```
// frontend/src/lib/utils.ts
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]): string {
  return twMerge(clsx(inputs));
}

// Example usage in an atelier component:
// <button className={cn("px-4 py-2 rounded-full", isSelected ? "bg-[#C9A84C] text-black" : "bg
```

3.3 Express REST Controller with Dependency Decoupling

```
// backend/src/controllers/product.controller.ts
import { Request, Response } from "express";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

export const getProducts = async (req: Request, res: Response) => {
  try {
    const products = await prisma.product.findMany({
      where: { deletedAt: null },
      include: { variants: true, images: true, category: true },
    });
    res.status(200).json({ success: true, count: products.length, data: products });
  } catch (error) {
    res.status(500).json({ success: false, error: "Failed to fetch products" });
  }
};
```

4. Configuration Guide & Diagnostics

4.1 TypeScript Configuration (tsconfig.json)

The monorepo enforces strict type safety:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

4.2 Next.js Configuration (next.config.ts)

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  reactStrictMode: true,
  images: {
    remotePatterns: [
      { protocol: "https", hostname: "**" },
      { protocol: "http", hostname: "localhost" },
    ],
  },
  // Ensure Node native modules and Prisma clients bundle seamlessly
  serverExternalPackages: ["@prisma/client"],
};

export default nextConfig;
```

4.3 Tailwind CSS v4 Engine Configuration

Tailwind v4 utilizes the new @import "tailwindcss"; directive directly within frontend/src/app/globals.css. Design tokens for the luxury atelier are declared as native CSS custom properties:

```
@import "tailwindcss";

@layer base {
  :root {
    --color-noir: #0D0D0D;
    --color-gold: #C9A84C;
    --color-parchment: #F9F6F0;
    --color-charcoal: #1F2022;
    --color-muted: #8E8D8A;
  }
}
```

4.4 ESLint 9 Flat Configuration (eslint.config.mjs)

Configured to extend Next.js core web vitals and React 19 hook guidelines:

- Enforces strict purity inside React useEffect hooks.
- Flags unused imports, variables, and untyped any castings.
- Guarantees zero lint errors during continuous integration builds.

3. REPOSITORY ARCHITECTURE & FILE MAP

1. Monorepo Structural Philosophy

NOIR & GOLD is organized as an npm workspace monorepo. This architectural approach delivers three critical engineering benefits:

1. Shared Domain Schemas: The database definitions (backend/prisma/schema.prisma) and SQLite database file (backend/prisma/dev.db) serve as the single source of truth for both the Next.js presentation layer and the Express microservice.
2. Unified Testing Pipeline: Automated tests can exercise full customer-to-admin workflows across all layers from a single test runner.
3. Operational Independence: The frontend and backend can be built, containerized, and deployed independently or operated concurrently during local development.

2. Complete Annotated Repository Tree

```
cloths store/
% % %  package.json                        # Monorepo root workspace config & scripts
% % %  package-lock.json                   # Lockfile for root and workspace packages
% % %  tsconfig.json                       # Monorepo base TypeScript settings
%
% % %  frontend/                           # Next.js 16 Web Application & Edge APIs
%    % % %  package.json                    # Frontend dependencies (React 19, Next 16, Prisma)
%    % % %  tsconfig.json                   # Frontend strict TypeScript configuration
%    % % %  next.config.ts                 # Next.js runtime configuration
%    % % %  eslint.config.mjs              # ESLint 9 configuration with React 19 rules
%    %
%    % % %  public/                        # Static public web assets
%    %    % % %  images/products/          # High-resolution catalog photography
%    %    % % %  uploads/                 # Administrative local multipart uploads
%    %
%    % % %  src/                           # Source code root
%    %    % % %  app/                      # Next.js App Router (pages and route handlers)
%    %    %    % % %  layout.tsx           # Root atelier layout (Navbar, Footer, Providers)
%    %    %    % % %  page.tsx             # Atelier landing page (Hero, Editorial, Drops)
%    %    %    % % %  globals.css          # Global styles, Tailwind v4 imports, theme variables
%    %    %    %
%    %    %    % % %  collection/          # Catalog discovery page
%    %    %    %    % % %  page.tsx        # Filterable product grid with category selectors
%    %    %    % % %  product/[slug]/      # Product detail page
%    %    %    %    % % %  page.tsx        # Interactive gallery, variant matrix, add-to-bag
%    %    %    % % %  cart/                # Shopping bag view
%    %    %    %    % % %  page.tsx        # Bag breakdown, subtotal, quantity adjustments
%    %    %    % % %  checkout/            # Multi-step checkout view
%    %    %    %    % % %  page.tsx        # Shipping form, zone charges, payment selection
%    %    %    % % %  orders/[id]/         # Customer order tracking page
%    %    %    %    % % %  page.tsx        # HMAC-protected guest tracking & courier details
%    %    %    % % %  admin/               # Administrative command center
%    %    %    %    % % %  page.tsx        # Admin dashboard (Products, Orders, Ledger, Exports)
%    %    %    %
... [remaining code truncated for layout brevity]
```

3. Deep-Dive: Responsibilities & Separation of Concerns

3.1 Frontend App Router (frontend/src/app)

- Primary Mission: Server-Side Rendered (SSR) storefront and secure administrative interface.

Why App Router? Next.js App Router collocates UI components with their backend route handlers (route.ts). For example, administrative product mutations reside directly in frontend/src/app/api/v1/admin/products/[id]/route.ts, enabling instant access to cookies, request headers, and server-side validation without cross-server hops.

3.2 Service Layer (frontend/src/lib)

- Primary Mission: Encapsulate pure business logic, database transactions, and security checks away from the HTTP transport layer.
- Why Decouple Services from Routes? By placing checkout logic in order-service.ts rather than inside the API route handler, the exact same logic can be executed, unit-tested, and stressed by automated test suites (tests/order-service.test.ts) without requiring a live HTTP server.

3.3 Backend Microservice (backend/src)

- Primary Mission: Standalone Express REST API operating on port 5000.
- Role in the Architecture: Provides a headless REST API for third-party integrations, mobile application consumers, or external microservices that need to query the catalog without loading Next.js page rendering pipelines.

---

4. Frontend API Routes vs. Express Backend Comparison

Architectural Aspect	Frontend Route Handlers (frontend/src/app/api/...)	Backend Express API (backend/src/...)
Port / Base URL	http://localhost:3000/api/v1	http://localhost:5000/api
Execution Environment	Next.js Server Runtime (Node.js)	Standalone Node.js HTTP Server
Primary Consumer	Next.js Storefront & Admin Dashboard	External clients, headless frontends, mobile apps
Authentication Model	HTTP-Only Session Cookies (admin-token)	API Keys / Bearer Tokens / Open CORS
Specialized Capabilities	Streaming CSV responses, multipart file handling	High-throughput JSON microservice endpoints
Database Access	Direct Prisma client (@/lib/prisma)	Direct Prisma client (@prisma/client)

4. FRONTEND ARCHITECTURE & APP ROUTER DEEP-DIVE

1. Next.js 16 App Router Mechanics

The NOIR & GOLD user interface is constructed using the Next.js 16 App Router (frontend/src/app), leveraging React 19's Server Components and Client Components to achieve fast initial page loads, optimal SEO indexing, and reactive client-side interactions.

1.1 Server Components vs. Client Components

In Next.js App Router, all components inside the app/ directory are React Server Components (RSC) by default unless marked with the "use client" directive.

- Server Components (e.g., app/layout.tsx, static page wrappers):
- Execute exclusively on the server at request time or build time.
- Can directly import server-only modules (such as database queries or file system utilities).
- Ship zero JavaScript to the browser bundle, drastically reducing initial payload sizes.
- Client Components (marked with "use client", e.g., ProductsManagerView.tsx, cart/page.tsx):
- Hydrated in the browser to enable DOM event listeners, state hooks (useState, useReducer), and lifecycle effects (useEffect, useCallback).
- Essential for interactive features: modal dialogs, variant selectors, drag-and-drop file uploaders, and dynamic forms.

```
flowchart TD
    subgraph Server Boundary
        Layout[app/layout.tsx - Server Component]
        Page[app/page.tsx - Server Component]
    end

    subgraph Hydration Boundary "use client"
        Nav[Navbar.tsx - Client Component]
        Hero[HeroSection.tsx - Client Component]
        Bag[CartDrawer.tsx - Client Component]
    end

    Layout --> Nav
    Layout --> Page
    Page --> Hero
    Nav --> Bag
```

2. Page Routes & Data Flow Specification

Route Path	Type	State / Hooks	Data Fetching Strategy	Rendered Output
/	Server / Client	Hero animation, featured carousel	Server-side catalog prefetch	Atelier landing page, curated drops, brand narrative
/collection	Client	Category filters, sort order, search	Client fetch /api/v1/products	Responsive multi-column grid of luxury silhouettes
/product/[slug]	Client	Active variant, color hex, quantity	Dynamic slug lookup via API	Multi-angle image gallery, size picker, add-to-bag
/cart	Client	Items, subtotals, quantity	LocalStorage reactive sync	Itemized bag summary, delivery threshold bar
/checkout	Client	Shipping address, zone, payment	Zone calculation API	Multi-step checkout form, COD / gateway selector
/orders/[id]	Client	Order status, delivery tracking	HMAC-authenticated fetch	Customer order status timeline and courier details

/admin

Client

Tabs (Products, Orders, Upload)

Authenticated session API

Administrative operations dashboard

3. Component Hierarchy & Organization

The UI is structured into modular component libraries within frontend/src/components:

```
src/components/
% % %  admin/                    # Restricted Administrative Components
%    % % %  AdminLayout.tsx        # Navigation header, tab switcher, staff sign-out
%    % % %  ProductsManagerView.tsx# Garment CRUD, soft-delete archive, feedback toasts
%    % % %  OrdersManagerView.tsx  # Order filtering drawer, courier dispatch, refund modal
%    % % %  VariantMatrixBuilder.tsx# Dynamic SKU generator, stock allocation, color picker
%    % % %  ImageUploader.tsx      # Drag-and-drop multipart upload zone with preview
% % %  cart/                     # Shopping Bag Components
%    % % %  CartDrawer.tsx         # Slide-over bag panel accessible from navbar
%    % % %  CartItemRow.tsx        # Individual garment row with quantity counter
% % %  checkout/                 # Order Finalization Components
%    % % %  ShippingZoneSelector.tsx# Dynamic delivery calculation by province
%    % % %  PaymentMethodRadio.tsx # Selection cards for COD, JazzCash, EasyPaisa
% % %  common/                   # Generic Atoms & Molecules
%    % % %  Modal.tsx             # Accessible dialog backdrop with keyboard escape
%    % % %  Badge.tsx             # Status pill (ACTIVE, SOLD OUT, PENDING, SHIPPED)
%    % % %  Spinner.tsx           # Animated luxury gold spinner
% % %  layout/                   # Persistent Shell Components
    % % %  Navbar.tsx            # Brand header, navigation links, bag counter
    % % %  Footer.tsx            # Atelier details, care guides, copyright
```

4. State Management Patterns

4.1 Shopping Bag State (Client-Side Persistence)

The shopping cart state is managed via a lightweight custom reactive hook that synchronizes with browser localStorage:

- State Structure:

```
interface CartItem {
  productId: string;
  variantId: string;
  title: string;
  slug: string;
  size: string;
  colorName: string;
  pricePkr: number;
  quantity: number;
  imageUrl: string;
  maxStock: number;
}
```

- Validation Guard: Prevents the user from incrementing the cart quantity beyond the variant's maxStock verified at the time of addition.

4.2 Administrative Form & Mutation State

Admin views utilize structured local state with clean feedback mechanisms:

- Asynchronous Cancellation Pattern (React 19 Safe):

```
  useEffect(() => {
    let ignore = false;
    async function loadData() {
      try {
        const res = await fetch("/api/v1/admin/products");
        const json = await res.json();
        if (!ignore && json.success) {
          setProducts(json.products);
        }
      } catch (err) {
        if (!ignore) showFeedback("error", "Failed to retrieve catalog");
      }
    }
    void loadData();
    return () => { ignore = true; };
  }, [showFeedback]);
```

5. Styling System: Tailwind CSS v4 & Haute Couture Design Tokens

The styling architecture is anchored on minimal elegance, luxury typography, and high-contrast editorial tones.

5.1 Color Palette Tokens

- Noir (#0D0D0D): Primary background, high-contrast headings, luxury framing.
- Champagne Gold (#C9A84C): Accent highlights, primary interactive buttons, badges.
- Soft Parchment (#F9F6F0): Light mode container backgrounds, crisp product cards.
- Charcoal Slate (#1F2022): Secondary buttons, card borders, subtle dividers.
- Muted Silver (#8E8D8A): Metadata, SKU codes, secondary captions.

5.2 Typography System

- Serif Display: Cinzel / Playfair Display for garment names, editorial quotes, and section titles.
- Sans-Serif Body: Inter / Montserrat for crisp pricing, specifications, and form fields.

6. Frontend Performance & Asset Optimization

1. Next.js <Image /> Component:
- Automatically converts source JPEGs into modern WebP/AVIF formats based on client browser support.
- Enforces responsive sizes attributes to prevent layout shift (CLS = 0).
- Implements blurred low-quality placeholders (LQIP) while high-resolution photography loads.
2. Code Splitting & Dynamic Imports:
- Heavy administrative views (VariantMatrixBuilder, OrdersManagerView) are lazy-loaded when the administrator accesses their respective tabs, keeping the customer bundle light.
3. Optimistic UI Updates:
- When administrators archive or restore garments, UI state updates optimistically before server round-trips complete, delivering instant responsiveness.

5. BACKEND ARCHITECTURE & EXPRESS REST API

1. Express Service Overview

The backend microservice (backend/) is built on Express 4.21.2 with TypeScript, executing on port 5000. It is engineered as a high-throughput, decoupled REST API capable of serving garment and category catalogs to diverse client surfaces without running through the Next.js server-rendering cycle.

```
flowchart TD
    Req[Incoming HTTP Request] --> CORS[CORS Middleware]
    CORS --> Morgan[Morgan Request Logger]
    Morgan --> Parser[Express JSON Body Parser]
    Parser --> Router[Express Router Mapping]
    Router --> Controller[Controller Logic: Product / Category]
    Controller --> Prisma[Prisma ORM Client Query]
    Prisma --> DB[(SQLite Database dev.db)]
    Prisma -.-> Controller
    Controller -.-> Res[JSON HTTP Response]
```

2. Middleware Chain & Execution Lifecycle

Requests entering the Express application traverse four standardized middleware layers configured in backend/src/app.ts:

1. CORS (Cross-Origin Resource Sharing):

```
app.use(cors({
  origin: process.env.CLIENT_URL || "http://localhost:3000",
  credentials: true,
}));
```

Enables the frontend Next.js application to safely dispatch client-side API requests to port 5000 while rejecting unauthorized third-party origins.

2. Morgan HTTP Logger:

```
app.use(morgan("dev"));
```

Outputs concise colored request logs (HTTP method, path, status code, response time in ms) to the terminal, essential for monitoring API throughput and diagnosing latency bottlenecks.

3. Express JSON Body Parser:

```
app.use(express.json());
```

Parses incoming JSON payloads with a standardized 10 MB payload limit to prevent denial-of-service through memory exhaustion.

4. Global Error-Handling Middleware:

Intersects unhandled controller exceptions, sanitizes error stack traces in production environments, and formats a consistent JSON error response:

```
{
  "success": false,
  "error": "Internal Server Error"
}
```

3. Controller Pattern & Separation of Concerns

The Express backend strictly enforces a three-tier separation of concerns:

- Routes Layer (backend/src/routes/): Purely maps HTTP verbs and URL paths to corresponding controller methods. No business logic resides here.
- Controller Layer (backend/src/controllers/): Extracts parameters from req.params, req.query, and req.body, validates inputs, invokes Prisma operations, and formats HTTP responses.
- Data Access Layer (backend/prisma/): Manages relational queries, foreign key relations, and soft-delete criteria via the generated Prisma client.

3.1 Product Controller Implementation

```
// backend/src/controllers/product.controller.ts
import { Request, Response } from "express";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

export const getProducts = async (req: Request, res: Response) => {
  try {
    const { category, search } = req.query;

    const products = await prisma.product.findMany({
      where: {
        deletedAt: null, // Soft-delete filter
        ...(category ? { category: { slug: String(category) } } : {}),
        ...(search ? {
          OR: [
            { title: { contains: String(search) } },
            { description: { contains: String(search) } },
          ],
        } : {}),
      },
      include: {
        variants: { where: { isActive: true } },
        images: { orderBy: { displayOrder: "asc" } },
        category: true,
      },
      orderBy: { createdAt: "desc" },
    });

    res.status(200).json({ success: true, count: products.length, data: products });
  } catch (error) {
    res.status(500).json({ success: false, error: "Failed to retrieve products" });
  }
};
```

3.2 Category Controller with Product Counters

```
// backend/src/controllers/category.controller.ts
import { Request, Response } from "express";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

export const getCategories = async (_req: Request, res: Response) => {
  try {
    const categories = await prisma.category.findMany({
      where: { deletedAt: null },
      include: {
        _count: {
          select: { products: { where: { deletedAt: null } } },
        },
      },
      orderBy: { name: "asc" },
    });

    res.status(200).json({ success: true, data: categories });
  } catch (error) {
    res.status(500).json({ success: false, error: "Failed to retrieve categories" });
  }
};
```

4. Prisma Connection Management & Query Optimization

- Singleton Pattern: In long-running Express processes, a single PrismaClient instance is instantiated and reused across the entire application lifecycle. This maintains an internal connection pool with SQLite, avoiding connection churn.
- Relational Eager Loading (include): Instead of issuing multiple sequential queries (N+1 query problem), Prisma bundles queries for products, variants, images, and categories into optimized SQL transactions.
- Soft-Delete Guardrails: Both controllers filter by deletedAt: null across primary queries and nested relational counts, guaranteeing that archived garments do not appear in public catalog responses.

5. Architectural Comparison: Next.js API Routes vs. Express REST API

Feature / Criterion	Next.js 16 Route Handlers	Express REST Backend
Directory Location	frontend/src/app/api/v1/	backend/src/
Runtime Port	3000 (Collocated with Next.js)	5000 (Dedicated Node process)
Primary Responsibility	Mutations, Auth, Admin Ops, Webhooks, CSV	Public Catalog Discovery & Category Feeds
Authentication System	Signed HTTP-Only Cookies (admin-token)	Stateless REST / Open Catalog
Response Format	NextResponse.json(...), ReadableStream	Express res.status().json(...)
Multipart Uploads	Web standard request.formData()	Multer / Busboy integration (extensible)
Streaming Capabilities	Native Web Streams (ReadableStream)	Node.js stream piping (res.write())

6. COMPLETE API REFERENCE

1. Overview & Authentication Standards

The NOIR & GOLD ecosystem exposes two API tiers:

1. Frontend Next.js Route Handlers: Serving at http://localhost:3000/api/v1 (Order checkout, customer lookups, payment webhooks, administrative operations).
2. Backend Express REST API: Serving at http://localhost:5000/api (Public product catalog and category listings).

Authentication Headers:

- Public Endpoints: No authentication headers required.
- Admin Endpoints: Require an authenticated HTTP-Only cookie named admin-token or an Authorization: Bearer <token> header, plus an X-Admin-Role: ADMIN | STAFF header for privileged operations.
- Payment Webhook: Requires x-payment-signature (HMAC-SHA256) and x-payment-timestamp (Unix epoch seconds).

---

2. Express Backend Catalog Endpoints (:5000)

2.1 List All Products

- Method: GET
- Path: /api/products
- Auth Required: None
- Query Parameters:
- category (optional string): Filter by category slug (e.g., pret, formal).
- search (optional string): Search across garment title and description.
- Success Response (200 OK):

```
{
  "success": true,
  "count": 1,
  "data": [
    {
      "id": "prod-001",
      "slug": "noir-velvet-sherwani",
      "title": "Noir Velvet Sherwani",
      "pricePkr": 85000,
      "isSoldOut": false,
      "category": { "name": "Formal", "slug": "formal" },
      "variants": [
        { "id": "var-001", "size": "M", "colorName": "Black", "stock": 4, "sku": "NVS-BLK-M"
      ],
      "images": [
        { "id": "img-001", "url": "/images/products/prod-001-primary.jpg", "isPrimary": true
      ]
    }
  ]
}
```

- Curl Example:

```
curl -X GET "http://localhost:5000/api/products?category=formal"
```

2.2 Get Product by Slug or ID

- Method: GET

Path: /api/products/:idOrSlug

- Auth Required: None
- Success Response (200 OK):

```
{
  "success": true,
  "data": {
    "id": "prod-001",
    "slug": "noir-velvet-sherwani",
    "title": "Noir Velvet Sherwani",
    "fabric": "Pure Handwoven Velvet",
    "pricePkr": 85000,
    "variants": [...]
  }
}
```

- Error Response (404 Not Found):

```
{ "success": false, "error": "Product not found" }
```

- Curl Example:

```
curl -X GET "http://localhost:5000/api/products/noir-velvet-sherwani"
```

2.3 List Categories

- Method: GET
- Path: /api/categories
- Auth Required: None
- Success Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "id": "cat-001",
      "name": "Formal",
      "slug": "formal",
      "_count": { "products": 4 }
    }
  ]
}
```

- Curl Example:

```
curl -X GET "http://localhost:5000/api/categories"
```

---

3. Customer Storefront Endpoints (:3000)

3.1 Place Customer Order

- Method: POST
- Path: /api/v1/orders
- Auth Required: None (Rate limited: Max 5 orders per 15 min per phone & IP)
- Request Body:

```
{
  "customer": {
    "fullName": "Fatima Ali",
    "email": "fatima@example.com",
    "phone": "+923001234567",
    "shippingAddress": "House 12, Street 4, F-7/2",
    "city": "Islamabad",
    "province": "Islamabad Capital Territory",
    "postalCode": "44000"
  },
  "items": [
    {
      "variantId": "var-001",
      "quantity": 1
    }
  ],
  "paymentMethod": "COD",
  "notes": "Please deliver before 5 PM"
}
```

- Success Response (201 Created):

```
{
  "success": true,
  "order": {
    "id": "ord-2026-9812",
    "orderStatus": "PENDING",
    "paymentStatus": "PENDING",
    "totalAmountPkr": 85000,
    "shippingFeePkr": 0,
    "token": "a1b2c3d4e5f6...",
    "trackingUrl": "/orders/ord-2026-9812?token=a1b2c3d4e5f6..."
  }
}
```

- Error Response (422 Unprocessable Content):

```
{
  "success": false,
  "error": { "code": "INSUFFICIENT_STOCK", "message": "Requested quantity exceeds available s
}
```

- Curl Example:

```
curl -X POST "http://localhost:3000/api/v1/orders" \
  -H "Content-Type: application/json" \
  -d '{"customer":{"fullName":"Fatima Ali","phone":"+923001234567","shippingAddress":"F-7","c
```

3.2 Guest Order Status Lookup

- Method: GET
- Path: /api/v1/orders/:id?token=:token
- Auth Required: Cryptographic HMAC token parameter matching the order
- Success Response (200 OK):

```
{
  "success": true,
  "order": {
    "id": "ord-2026-9812",
    "orderStatus": "CONFIRMED",
    "courierName": "TCS Express",
    "trackingNumber": "TCS-998822",
    "items": [...]
  }
}
```

- Error Response (403 Forbidden):

```
{ "success": false, "error": { "code": "INVALID_TOKEN", "message": "Access denied." } }
```

- Curl Example:

```
curl -X GET "http://localhost:3000/api/v1/orders/ord-2026-9812?token=VALID_HMAC_TOKEN"
```

---

4. Payment Gateway Webhook (:3000)

4.1 Payment Gateway Webhook Route

- Method: POST
- Path: /api/v1/payments/webhook
- Auth Required: HMAC-SHA256 signature verification (x-payment-signature)
- Headers:
- x-payment-signature: Hex HMAC-SHA256 signature calculated over the raw body
- x-payment-timestamp: Unix epoch seconds (must be within 300 seconds of current time)
- Request Body:

```
{
  "eventType": "payment.captured",
  "orderId": "ord-2026-9812",
  "transactionId": "txn-jazz-9912",
  "amountPkr": 85000,
  "currency": "PKR",
  "status": "PAID"
}
```

- Success Response (200 OK):

```
{ "success": true, "processed": true, "orderId": "ord-2026-9812" }
```

- Error Responses:
- 401 Unauthorized: Missing or invalid HMAC signature
- 408 Request Timeout: Webhook timestamp older than 300 seconds (replay attack prevention)
- 500 Internal Server Error: PAYMENT_WEBHOOK_SECRET unset in production (fail-closed)
- Curl Example:

```
curl -X POST "http://localhost:3000/api/v1/payments/webhook" \
  -H "Content-Type: application/json" \
  -H "x-payment-signature: VALID_HMAC_HEX" \
  -H "x-payment-timestamp: 1776470400" \
  -d '{"eventType":"payment.captured","orderId":"ord-2026-9812","status":"PAID"}'
```

---

5. Administrative Endpoints (:3000)

5.1 Admin Session Authentication

- Method: POST
- Path: /api/v1/admin/auth/login
- Auth Required: None (Rate-limited: Max 5 failed attempts per 15 min per email)
- Request Body:

```
{
  "email": "director@noirandgold.com",
  "password": "atelier-secret-password",
  "secretKey": "master-passphrase"
}
```

- Success Response (200 OK):
- Sets HTTP-Only cookie: admin-token=<session_token>

```
{ "success": true, "role": "ADMIN", "user": { "email": "director@noirandgold.com" } }
```

5.2 Product Mutation & Soft Delete

- Method: PATCH / DELETE
- Path: /api/v1/admin/products/:id
- Auth Required: Valid session (ADMIN or STAFF role)
- PATCH Body (Update Garment & Variants):

```
{
  "title": "Noir Velvet Sherwani (2026 Edition)",
  "pricePkr": 89000,
  "variants": [
    { "size": "M", "colorName": "Black", "stock": 6, "sku": "NVS-BLK-M-26", "pricePkr": 89000
  ]
}
```

- PATCH Body (Restore Archived Product):

```
{ "action": "RESTORE" }
```

- DELETE Method (Soft-Delete / Archive):
- Sets deletedAt = new Date() and isSoldOut = true.

5.3 Process Refund

- Method: POST
- Path: /api/v1/admin/orders/:id/refund
- Auth Required: ADMIN Role required
- Request Body:

```
{
  "amountPkr": 25000,
  "reason": "Customer requested size cancellation prior to shipping",
  "refundMethod": "GATEWAY"
}
```

- Success Response (200 OK):

```
{
  "success": true,
  "refund": {
    "id": "ref-4412",
    "amountPkr": 25000,
    "status": "COMPLETED",
    "remainingRefundablePkr": 60000
  }
}
```


- Error Response (400 Bad Request):


```
{
  "success": false,
  "error": { "code": "CAPACITY_EXCEEDED", "message": "Refund amount exceeds captured order ba
}
```


5.4 Streaming CSV Financial Export

- Method: GET
- Path: /api/v1/admin/orders/export?status=ALL&startDate=2026-01-01
- Auth Required: Valid session (ADMIN or STAFF)
- Response Headers:
- Content-Type: text/csv; charset=utf-8
- Content-Disposition: attachment; filename="noir-gold-orders-export-...csv"
- Response Format: RFC-4180 streaming CSV output.

7. DATABASE DESIGN & PRISMA ORM DEEP-DIVE

1. Relational Architecture & 14 Models

The NOIR & GOLD persistence layer is modeled in backend/prisma/schema.prisma and realized in SQLite (backend/prisma/dev.db). The architecture features 14 synchronized relational tables designed for financial precision, state machine tracking, and audit durability.

```
erDiagram
    CATEGORY ||--o{ PRODUCT : contains
    PRODUCT ||--o{ PRODUCT_VARIANT : defines
    PRODUCT ||--o{ PRODUCT_IMAGE : displays
    PRODUCT ||--o{ INVENTORY_RESERVATION : holds
    PRODUCT_VARIANT ||--o{ ORDER_ITEM : fulfilled_by
    PRODUCT_VARIANT ||--o{ INVENTORY_RESERVATION : reserves
    PRODUCT_VARIANT ||--o{ INVENTORY_AUDIT_LOG : tracks
    ORDER ||--o{ ORDER_ITEM : includes
    ORDER ||--o{ PAYMENT_ATTEMPT : processes
    ORDER ||--o{ REFUND : credits
    ORDER ||--o{ ORDER_AUDIT_LOG : audits
    PAYMENT_ATTEMPT ||--o{ PAYMENT_EVENT : logs
    SHIPPING_ZONE ||--o{ ORDER : applies_to
    ADMIN_USER ||--o{ SECURITY_AUDIT_LOG : performs
```

2. Model-by-Model Breakdown

2.1 Catalog & Product Models

Category

- id (String, PK, cuid/uuid)
- name (String, e.g., "Bridal", "Formal", "Pret")
- slug (String, Unique, URL identifier)
- description (String?)
- deletedAt (DateTime?, Soft delete index)
- createdAt / updatedAt (DateTime)

Product

- id (String, PK)
- slug (String, Unique)
- title (String)
- description (String)
- fabric (String, artisanal textile specification)
- pricePkr (Int, integer representation in PKR)
- salePricePkr (Int?)
- isSoldOut (Boolean, default false)
- categoryId (String, FK -> Category.id)
- deletedAt (DateTime?, indexed for soft-delete filtering)
- createdAt / updatedAt (DateTime)

ProductVariant

- id (String, PK)
- productId (String, FK -> Product.id, Cascade on delete)

size (String, e.g., "XS", "S", "M", "L", "XL", "Custom")
- colorName (String, e.g., "Obsidian Noir", "Champagne Gold")
- colorHex (String, e.g., "#0D0D0D")
- sku (String, Unique, e.g., "NVS-BLK-M")
- stock (Int, physical inventory on hand)
- pricePkr (Int?)
- isActive (Boolean, default true)

ProductImage
- id (String, PK)
- productId (String, FK -> Product.id)
- url (String, local path or S3 URL)
- alt (String?)
- isPrimary (Boolean, default false)
- displayOrder (Int, default 0)

2.2 Order & Fulfillment Models

Order
- id (String, PK, e.g., "ord-2026-9812")
- customerName (String)
- customerEmail (String?)
- customerPhone (String, indexed for rate limiting and lookup)
- shippingAddress (String)
- city (String)
- province (String)
- orderStatus (String: PENDING, CONFIRMED, PROCESSING, SHIPPED, DELIVERED, CANCELLED)
- paymentStatus (String: PENDING, PAID, FAILED, REFUNDED, PARTIALLY_REFUNDED)
- paymentMethod (String: COD, JAZZCASH, EASYPAISA, BANK_TRANSFER)
- subtotalPkr (Int)
- shippingFeePkr (Int)
- totalAmountPkr (Int)
- courierName (String?, e.g., "TCS", "Leopards")
- trackingNumber (String?)
- tokenHash (String, SHA256 hash for guest HMAC tracking)
- deletedAt (DateTime?, soft-delete index)
- createdAt / updatedAt (DateTime)

OrderItem
- id (String, PK)
- orderId (String, FK -> Order.id)
- variantId (String, FK -> ProductVariant.id)
- productTitle (String, snapshot at purchase time)
- variantSku (String, snapshot)
- size (String)
- colorName (String)
- unitPricePkr (Int)
- quantity (Int)
- totalPricePkr (Int)

ShippingZone
- id (String, PK)

name (String, e.g., "Sindh", "Punjab", "KPK / Balochistan")
- baseRatePkr (Int)
- freeDeliveryThresholdPkr (Int)
- estimatedDays (String)

2.3 Financial & State Ledger Models

PaymentAttempt
- id (String, PK)
- orderId (String, FK -> Order.id)
- attemptKey (String, Unique per order for idempotency)
- provider (String, "JAZZCASH", "EASYPAISA", "STRIPE")
- amountPkr (Int)
- currency (String, default "PKR")
- status (String: INITIATED, PENDING_GATEWAY, CAPTURED, FAILED, CANCELLED)
- providerReference (String?)
- createdAt / updatedAt (DateTime)

PaymentEvent
- id (String, PK)
- paymentAttemptId (String, FK -> PaymentAttempt.id)
- providerEventId (String, Unique)
- eventType (String, e.g., "payment.captured", "payment.failed")
- payload (String, raw JSON for immutable auditing)
- processedAt (DateTime, default now())

Refund
- id (String, PK)
- orderId (String, FK -> Order.id)
- gatewayRefundId (String?)
- amountPkr (Int)
- reason (String)
- refundMethod (String: "GATEWAY", "BANK_TRANSFER", "STORE_CREDIT")
- status (String: PENDING, COMPLETED, FAILED)
- processedBy (String?, admin user id)
- createdAt / updatedAt (DateTime)

InventoryReservation
- id (String, PK)
- productId (String, FK -> Product.id)
- productVariantId (String, FK -> ProductVariant.id)
- quantity (Int)
- reservationKey (String, Unique checkout idempotency token)
- expiresAt (DateTime, time-bounded checkout lock)
- isConsumed (Boolean, default false)
- isReleased (Boolean, default false)

2.4 Auditing & Governance Models

SecurityAuditLog
- id (String, PK)
- action (String, e.g., "PRICE_CHANGE", "ADMIN_LOGIN", "REFUND_ISSUED", "PRODUCT_ARCHIVED")

targetResource (String, e.g., "Product:prod-001", "Order:ord-9812")
- performedBy (String, admin email or "SYSTEM")
- ipAddress (String?)
- metadata (String?, JSON snapshot of prior and new states)
- createdAt (DateTime, default now(), immutable)

OrderAuditLog & InventoryAuditLog

- Immutable records capturing chronological transitions in order statuses and physical variant stock levels.

---

3. Soft-Delete Architecture

In enterprise e-commerce, physical row deletion (DELETE FROM table) is strictly prohibited for products and categories because historical customer orders rely on those records for receipts, invoices, and accounting audits.

Implementation Standard:

1. Every major entity (Product, Category, Order) possesses a nullable deletedAt DateTime? column.
2. An index is declared on the column (@@index([deletedAt])) for zero-overhead query filtering.
3. When an entity is archived:

```
  await prisma.product.update({
    where: { id: productId },
    data: { deletedAt: new Date(), isSoldOut: true },
  });
```

4. When queried across public and admin interfaces:

```
  where: { deletedAt: null }
```

5. Soft-deleted entities can be instantly restored without data loss:

```
  await prisma.product.update({
    where: { id: productId },
    data: { deletedAt: null, isSoldOut: false },
  });
```

---

4. 10 Essential Real-World Prisma Query Patterns

Query 1: Eager-Loaded Catalog with Soft-Delete Filter

```
const products = await prisma.product.findMany({
  where: { deletedAt: null, isSoldOut: false },
  include: {
    category: true,
    variants: { where: { isActive: true } },
    images: { orderBy: { displayOrder: "asc" } },
  },
});
```

Query 2: Atomic Inventory Decrement with Optimistic Locking

```
const updated = await prisma.productVariant.updateMany({
  where: { id: variantId, stock: { gte: quantityToBuy } },
  data: { stock: { decrement: quantityToBuy } },
});
```

```
  if (updated.count === 0) throw new Error("INSUFFICIENT_STOCK");
```

Query 3: Multi-Table Transaction (Order + Items + Reservation)

```
const result = await prisma.$transaction(async (tx) => {
  const order = await tx.order.create({ data: orderData });
  await tx.orderItem.createMany({ data: itemsData });
  await tx.inventoryReservation.create({ data: reservationData });
  return order;
});
```

Query 4: Aggregate Refund Capacity Verification

```
const aggregate = await prisma.refund.aggregate({
  where: { orderId, status: "COMPLETED" },
  _sum: { amountPkr: true },
});
const totalRefunded = aggregate._sum.amountPkr || 0;
```

Query 5: Security Audit Log Ingestion

```
await prisma.securityAuditLog.create({
  data: {
    action: "ADMIN_LOGIN_SUCCESS",
    targetResource: "AdminSession",
    performedBy: adminEmail,
    ipAddress: clientIp,
    metadata: JSON.stringify({ role: "ADMIN", userAgent }),
  },
});
```

Query 6: Check Active Idempotent Payment Attempt

```
const attempt = await prisma.paymentAttempt.findUnique({
  where: { orderId_attemptKey: { orderId, attemptKey } },
});
```

Query 7: Filter Orders for CSV Export with Date Range

```
const orders = await prisma.order.findMany({
  where: {
    deletedAt: null,
    orderStatus: statusFilter !== "ALL" ? statusFilter : undefined,
    createdAt: { gte: startDate, lte: endDate },
  },
  include: { items: true, refunds: true },
  orderBy: { createdAt: "desc" },
});
```

Query 8: Soft-Delete Product and Audit

```
await prisma.$transaction([
  prisma.product.update({ where: { id }, data: { deletedAt: new Date() } }),
  prisma.securityAuditLog.create({ data: { action: "PRODUCT_ARCHIVE", targetResource: `Product:
]);
```

Query 9: Restore Soft-Deleted Product

```
await prisma.product.update({
  where: { id },
  data: { deletedAt: null, isSoldOut: false },
});
```

Query 10: Find and Release Expired Reservations

```
const expired = await prisma.inventoryReservation.findMany({
  where: { expiresAt: { lt: new Date() }, isConsumed: false, isReleased: false },
});
```

8. AUTHENTICATION, AUTHORIZATION & SECURITY ARCHITECTURE

1. Security Philosophy & Defense-in-Depth

The NOIR & GOLD platform handles high-value financial transactions, sensitive customer delivery data, and privileged catalog pricing. Security is structured on a multi-layered Defense-in-Depth model:

```
flowchart TD
    Client[Incoming Request] --> WAF[Network & Origin Validation]
    WAF --> RL[Dual Rate Limiters: IP & Phone]
    RL --> CSRF[CSRF & Sec-Fetch-Site Guard]
    CSRF --> Auth[Session Cookie Authentication]
    Auth --> RBAC[Role-Based Access Control: ADMIN vs STAFF]
    RBAC --> Service[Parameterized Business Logic]
    Service --> Audit[Security Audit Ingestion]
    Audit --> DB[(Encrypted SQLite Database)]
```

---

2. Administrative Authentication & Session Cookies

2.1 Credential Verification & Session Creation

Administrative access requires three credentials:

1. Registered Admin Email
2. Password Hash
3. Master Secret Key (ADMIN_SESSION_SECRET)

When authenticated via /api/v1/admin/auth/login:

- The server generates an HMAC-SHA256 signed session payload encoding: email, role, issuedAt, and expiresAt.
- The session token is transmitted to the client inside an HTTP-Only cookie:

```
  response.cookies.set("admin-token", token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    path: "/",
    maxAge: 60 * 60 * 8, // 8 hours duration
  });
```

2.2 Security Benefits of HTTP-Only Cookies

- Zero XSS Token Exposure: Client-side JavaScript running in the browser cannot read or copy the admin-token cookie via document.cookie.
- Automatic Browser Attachment: Subsequent requests from the admin dashboard automatically transmit the cookie across all origin requests.

---

3. Role-Based Access Control (RBAC): ADMIN vs. STAFF

The system defines two administrative tiers in frontend/src/lib/admin-auth-guard.ts:

Permission / Operation	ADMIN Role	STAFF Role	Implementation Guard
View Orders & Fulfillment	' Allowed	' Allowed	requireAdminOrStaff(req)

Update Courier &
Tracking

' Allowed

' Allowed

requireAdminOrStaff(req)

Permission / Operation	ADMIN Role	STAFF Role	Implementation Guard
View Garment Catalog	' Allowed	' Allowed	requireAdminOrStaff(req)
Edit Inventory Quantities	' Allowed	' Allowed	requireAdminOrStaff(req)
Upload Product Photography	' Allowed	' Allowed	requireAdminOrStaff(req)
Download Financial CSV	' Allowed	' Allowed	requireAdminOrStaff(req)
Create New Garments	' Allowed	'L Forbidden (403)	requireAdminRole(req)
Modify Base Garment Prices	' Allowed	'L Forbidden (403)	requireAdminRole(req)
Soft-Delete / Archive Garments	' Allowed	'L Forbidden (403)	requireAdminRole(req)
Issue Financial Refunds	' Allowed	'L Forbidden (403)	requireAdminRole(req)

4. Payment Gateway Webhook Security

Payment webhooks (/api/v1/payments/webhook) execute state changes (e.g., marking orders as PAID). Consequently, they must withstand tampering, replay attacks, and spoofing:

4.1 Elimination of Insecure Fallback & Fail-Closed Enforcement

```
const secret = process.env.PAYMENT_WEBHOOK_SECRET;
if (!secret) {
  if (process.env.NODE_ENV === "production") {
    console.error("FATAL: PAYMENT_WEBHOOK_SECRET is not configured.");
    return NextResponse.json(
      { success: false, error: { code: "WEBHOOK_CONFIG_ERROR", message: "Webhook secret missing
      { status: 500 }
    );
  }
}
```

4.2 HMAC-SHA256 Signature Verification

The webhook calculates the expected HMAC over the raw request payload:

```
const hmac = crypto.createHmac("sha256", secret);
hmac.update(rawBody);
const expectedSignature = hmac.digest("hex");

// Timing-safe comparison prevents timing side-channel attacks
const isValid = crypto.timingSafeEqual(
  Buffer.from(providedSignature, "hex"),
  Buffer.from(expectedSignature, "hex")
);
```

4.3 300-Second Timestamp Freshness Window (Anti-Replay)

```
const webhookTime = parseInt(req.headers.get("x-payment-timestamp") || "0", 10);
const currentTime = Math.floor(Date.now() / 1000);

if (Math.abs(currentTime - webhookTime) > 300) {
  return NextResponse.json(
    { success: false, error: { code: "REPLAY_DETECTED", message: "Timestamp expired." } },
    { status: 408 }
  );
}
```

5. Rate Limiting Architecture

To protect against credential stuffing and checkout denial-of-service, the platform enforces memory-bounded rate limiters:

5.1 Dual Checkout Limiter (Phone & IP)

Protects the public /api/v1/orders endpoint:

- Phone Limiter: Maximum 5 order placements per 15 minutes per customer phone number.
- IP Limiter: Maximum 10 order placements per 15 minutes per client IP address.
- Behavior on Violation: Returns HTTP 429 Too Many Requests with a Retry-After: 900 header.

5.2 Admin Login Brute-Force Limiter

Protects /api/v1/admin/auth/login:

- Maximum 5 failed login attempts per 15 minutes per target email address.
- Resets upon successful authentication.

6. CSRF Defense & Origin Validation

For state-changing mutations (POST, PATCH, DELETE):

1. Origin Verification: The Origin or Referer header must match the authorized host (localhost:3000 or production domain).
2. Sec-Fetch-Site Inspection: Modern browsers send Sec-Fetch-Site. The guard rejects requests if Sec-Fetch-Site: cross-site, neutralizing Cross-Site Request Forgery (CSRF).

7. SQL Injection Prevention & Input Sanitization

- Prisma Parameterized Queries: All database queries utilize Prisma's strongly typed query engine, which automatically escapes and parameterizes all inputs. Malicious strings like ' OR '1'='1 are treated strictly as literal string values, rendering SQL injection impossible.
- Payload Clamping: Checkout quantity requests are clamped between 1 and 10 units per variant, preventing integer overflow or negative inventory attacks.
- Multipart Upload Sanitization: Uploaded filenames are stripped of path-traversal sequences (../) and replaced with cryptographically random UUIDs (uuid.v4() + ext).

8. Security Audit Logging (SecurityAuditLog)

Every administrative or high-impact operational event is immutably appended to SecurityAuditLog:

- Admin sign-in (successful and failed attempts)
- Product price changes (capturing old price and new price in metadata)
- Soft-delete archiving and restorations
- Refund issuance (amount, reason, remaining balance)
- Webhook signature failures

9. E-COMMERCE BUSINESS WORKFLOWS & STATE MACHINES

1. Order Lifecycle State Machine

Orders in NOIR & GOLD progress through a deterministic, strictly enforced finite state machine. Transitions that violate business logic (e.g., jumping from PENDING directly to DELIVERED) are rejected by admin-order-update-service.ts.

```
stateDiagram-v2
    [*] --> PENDING: Customer Places Order
    PENDING --> CONFIRMED: Admin Verifies / Payment Captured
    PENDING --> CANCELLED: Customer or Admin Cancels
    CONFIRMED --> PROCESSING: Atelier Tailoring / Packaging
    CONFIRMED --> CANCELLED: Pre-shipment Cancellation
    PROCESSING --> SHIPPED: Courier Assigned & Dispatched
    PROCESSING --> CANCELLED: Exceptional Cancellation
    SHIPPED --> DELIVERED: Courier Confirms Receipt
    DELIVERED --> [*]
    CANCELLED --> [*]
```

Transition Rules & Restock Invariants:

1. PENDING -> CONFIRMED: Validates inventory reservation or captured payment.
2. CONFIRMED -> PROCESSING: Garment enters artisanal preparation.
3. PROCESSING -> SHIPPED: Requires courier metadata (courierName and trackingNumber). If missing, the transition fails.
4. Transition to CANCELLED:

- Atomic Restock: Restores the exact physical inventory of each variant.
- Reservation Release: Flags any active InventoryReservation as released to prevent duplicate restocks.
- Idempotency Guarantee: Cancelling an already cancelled order is a no-op and will never double-restock inventory.

---

2. Payment Lifecycle State Machine

```
stateDiagram-v2
    [*] --> PENDING: Order Created
    PENDING --> PAID: Webhook Captured / COD Collected
    PENDING --> FAILED: Gateway Rejection / Expired
    PAID --> PARTIALLY_REFUNDED: Partial Return Processed
    PAID --> REFUNDED: Full Order Refunded
    PARTIALLY_REFUNDED --> REFUNDED: Balance Refunded
    FAILED --> [*]
    REFUNDED --> [*]
```

- PENDING: Default state for all newly created orders (both COD and gateway).
- PAID: Set when an HMAC webhook reports payment.captured or when an admin marks a COD order as collected upon delivery.
- PARTIALLY_REFUNDED: Set when an administrative refund is processed for an amount strictly less than totalAmountPkr.
- REFUNDED: Set when cumulative refunds match 100% of the captured sum.

---

3. Inventory Reservation & Stock Holding

To prevent overselling during flash atelier drops, the platform implements Two-Phase Inventory Protection:

```
sequenceDiagram
    autonumber
    actor Customer
    participant CheckoutService as Checkout Service
    participant Prisma as Prisma Transaction
    participant DB as SQLite Database

    Customer->>CheckoutService: Submit Checkout (Variant, Qty)
    CheckoutService->>Prisma: Begin Transaction
    Prisma->>DB: Atomic Decrement stock WHERE stock >= Qty
    alt Insufficient Stock
        DB-->>Prisma: Updated Rows = 0
        Prisma-->>CheckoutService: Throw INSUFFICIENT_STOCK
        CheckoutService-->>Customer: Order Rejected
    else Stock Available
        DB-->>Prisma: Stock Decremented
        Prisma->>DB: Insert Order & OrderItems
        Prisma->>DB: Insert InventoryReservation (TTL = 15m)
        Prisma-->>CheckoutService: Commit Transaction
        CheckoutService-->>Customer: Order Placed (Reservation Held)
    end
```

Reservation States:

- Active Hold: isConsumed = false, isReleased = false, expiresAt > now(). Stock is deducted from the physical counter.
- Consumed: When the order transitions to CONFIRMED or SHIPPED, the reservation is finalized.
- Released (Restocked): If an order is cancelled or times out, the reservation is released and variant stock is restored.

---

4. Complete Checkout Workflow

1. Cart Review & Payload Submission: The browser submits recipient details, variant IDs, and quantities to /api/v1/orders.
2. Rate Limit Gate: Dual checkout limiter verifies IP and phone request frequencies.
3. Price & Shipping Recalculation: The server does not trust prices submitted by the client. It queries the database for active prices and evaluates destination shipping zone thresholds:

$$\text{Final Total} = \sum (\text{Variant Base Price} \times \text{Qty}) + \text{Zone Shipping Fee}$$

If $\sum \text{Subtotal} \ge \text{Free Delivery Threshold}$, shipping is set to PKR 0.

4. Atomic Transaction: Creates Order, OrderItem records, decrements variant inventory, and generates an HMAC lookup token.
5. Customer Redirect: Returns HTTP 201 with the order tracking URL.

---

5. Refund Processing Workflow

1. Admin Initiation: An administrator submits a refund request for an order with a reason and amount in PKR.
2. RBAC Verification: admin-auth-guard ensures the user possesses the ADMIN role.
3. Capacity Ledger Check:

$$\text{Max Refundable} = \text{Captured Amount} - \sum \text{Prior Completed Refunds}$$

If $\text{Requested Amount} > \text{Max Refundable}$, the transaction is rejected with CAPACITY_EXCEEDED.

4. Atomic Ledger Execution:

- Appends a new Refund row with status COMPLETED.
- Updates Order.paymentStatus to PARTIALLY_REFUNDED or REFUNDED.
- Appends an immutable record to SecurityAuditLog.

---

6. RFC-4180 Streaming CSV Export Workflow

```
flowchart LR
    A[Admin Filter Criteria] --> B[GET /api/v1/admin/orders/export]
    B --> C[Admin Auth Guard]
    C --> D[Prisma Batch Query Orders]
    D --> E[Streaming CSV Pipeline]
    E --> F[Escape Special Chars RFC-4180]
    F --> G[Browser File Download Stream]
```

- RFC-4180 Compliance: Fields containing commas, double quotes, or newlines are wrapped in quotation marks. Existing quotes are escaped as "".
- Memory Efficiency: Large order volumes are processed via Node streams, piping data directly to the HTTP response without loading millions of rows into server RAM.
- Granular Filters: Supports exporting by date ranges (startDate, endDate), payment status, and order status.

10. DEVELOPER ENVIRONMENT SETUP & CLI REFERENCE

1. System Prerequisites

Before launching the NOIR & GOLD development environment, ensure your system meets the following specifications:

- Node.js: v20.17.0 or higher (LTS recommended). Check with node -v.
- npm: v10.0.0 or higher. Check with npm -v.
- Git: Installed and available on system PATH.
- Operating System: Windows 10/11 (PowerShell), macOS, or Linux.

---

2. Step-by-Step Installation & Setup

Step 1: Clone Repository

```
git clone <repository_url>
cd "cloths store"
```

Step 2: Install Monorepo Dependencies

```
npm install
```

Step 3: Configure Environment Variables

Create .env in both frontend/ and backend/ directories.

frontend/.env.local:

```
# Database Connection (Absolute or Relative SQLite path)
DATABASE_URL="file:../../backend/prisma/dev.db"

# Admin Authentication & Session Passphrase
ADMIN_SESSION_SECRET="noir-gold-super-secure-production-passphrase-2026"

# Payment Webhook Security (HMAC-SHA256)
PAYMENT_WEBHOOK_SECRET="whsec_noir_and_gold_development_secret_key_8899"

# Public API URLs
NEXT_PUBLIC_API_URL="http://localhost:3000/api/v1"
NEXT_PUBLIC_BACKEND_URL="http://localhost:5000/api"

# Optional Cloud Storage (Leave blank for local disk fallback)
AWS_ACCESS_KEY_ID=""
AWS_SECRET_ACCESS_KEY=""
AWS_REGION="auto"
AWS_S3_BUCKET_NAME=""
```

backend/.env:

```
PORT=5000
DATABASE_URL="file:./dev.db"
CLIENT_URL="http://localhost:3000"
NODE_ENV="development"
```

Step 4: Synchronize Database & Seed Catalog

```
npm run backend:db:push
npm run backend:db:seed
```

Step 5: Launch Development Servers

Open two terminal windows:

```
# Terminal 1: Backend Express REST API (:5000)
npm run backend:dev

# Terminal 2: Frontend Next.js Atelier (:3000)
npm run frontend:dev
```

3. Monorepo Script Reference

All scripts can be invoked from the root directory or targeted directly via npm workspaces:

Script Command	Workspace Target	Action Performed
npm run frontend:dev	frontend	Launches Next.js dev server with Turbopack on port 3000
npm run frontend:build	frontend	Compiles optimized Next.js production build
npm run frontend:lint	frontend	Executes ESLint 9 checks across all pages and components
npm run frontend:typecheck	frontend	Runs tsc --noEmit to verify type safety
npm test	frontend	Executes full native test runner across all 122 tests
npm run backend:dev	backend	Runs Express server with tsx watch for auto-reloading
npm run backend:build	backend	Compiles Express TypeScript code to dist/app.js
npm run backend:start	backend	Runs compiled Express server using standard Node.js
npm run backend:db:push	backend	Pushes Prisma schema modifications directly to SQLite
npm run backend:db:seed	backend	Populates database with artisanal garments and shipping zones

4. Database Administration & Inspection

Visualizing the Database with Prisma Studio

To inspect tables, edit records, or review foreign keys visually:

```
cd backend
npx prisma studio
```

Opens an interactive web GUI at http://localhost:5555.

Regenerating Prisma Client Types

Whenever you edit backend/prisma/schema.prisma:

```
# Push schema changes to disk
npm run backend:db:push

# Regenerate client definitions for frontend
cd frontend
npx prisma generate --schema=../backend/prisma/schema.prisma
```

5. Troubleshooting & FAQ

Problem 1: Port Already in Use (EADDRINUSE: 3000 or 5000)

- Cause: A previous instance of Next.js or Express is still running in the background.
- Solution (Windows PowerShell):

```
# Find PID occupying port 3000
Get-NetTCPConnection -LocalPort 3000 | Select-Object OwningProcess
# Kill process by PID
Stop-Process -Id <PID> -Force
```

Problem 2: Prisma Database File Locked (dev.db-journal)

- Cause: SQLite enters write lock when a test or crashed process held an uncommitted transaction.
- Solution: Close open terminals, terminate any zombie Node processes (Stop-Process -Name node -Force), and delete temporary dev.db-journal files if present.

Problem 3: PowerShell Script Execution Policy Error

- Error: File ... cannot be loaded because running scripts is disabled on this system.
- Solution: Run PowerShell as Administrator and execute:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Problem 4: Next.js Hydration Warning

- Cause: Browser extensions (e.g., password managers, translators) injecting extra HTML tags into the DOM before hydration.
- Solution: Ensure suppressHydrationWarning is added to <body> in frontend/src/app/layout.tsx.

11. TESTING STRATEGY & TEST SUITE GUIDE

1. Testing Philosophy: Zero-Dependency Architecture

NOIR & GOLD utilizes the native Node.js test runner (node:test) and strict assertion library (node:assert/strict) executed directly via tsx.

Why Native node:test Over Jest / Mocha / Vitest?

1. Zero External Dependencies: Built directly into the Node.js runtime. No brittle Babel plugins, transformer configurations, or version drift.
2. Sub-Second Execution: Bypasses overhead from heavy test runners. The entire 122-test integration suite executes in under 15 seconds.
3. Native TypeScript Support: Paired with tsx, tests run TypeScript directly without pre-compilation.

2. Comprehensive Test Suite Inventory

The project features 122 automated tests across six specialized suites in frontend/tests/:

Test Suite File	Coverage Focus	Test Count	Key Invariants Verified
order-service.test.ts	Order Creation & Concurrency	23	Non-overselling under load, atomic rollback on failure, token hashing
payment-attempt.test.ts	Payment State Machine	14	State transitions, idempotency key collisions, replay prevention
refund-ledger.test.ts	Financial Ledger & Capacity	10	Cumulative capacity checks, gateway refund events, audit logging
inventory-reservation.test.ts	Stock Holds & TTL	7	Reservation expiration, atomic release on cancellation, no double-restock
security-audit.test.ts	Security & Rate Limits	8	Phone/IP rate limiting, CSRF origin checks, SQL injection resistance
admin-orders-export.test.ts	Financial CSV Export	6	RFC-4180 quotation escaping, date filtering, status filters
admin-upload.test.ts	Multipart File Upload	4	5 MB file ceiling, MIME validation, collision-resistant UUIDs
e2e-all-phases.test.ts	Master E2E Lifecycle	50	Complete customer-to-admin lifecycle across all 5 remediation phases
Total Test Count	Full System Coverage	122	100% Pass Rate (0 Failures)

3. Database Isolation Strategy (mkdtempSync)

Running automated database tests against the live development database (dev.db) creates severe risks: tests leave dirty rows, foreign key collisions occur, and local seed data is corrupted.

To solve this, tests implement dynamic per-run database cloning:

```
// Pattern used in test suites:
import fs from "node:fs";
import os from "node:os";
import path from "node:path";

function setupIsolatedTestDb(): string {
  const tmpDir = fs.mkdtempSync(path.join(os.tmpdir(), "noir-gold-test-"));
  const tmpDbPath = path.join(tmpDir, "test.db");
  const sourceDb = path.resolve(__dirname, "../../backend/prisma/dev.db");

  // Copy pristine physical database to isolated RAM/Temp disk
  fs.copyFileSync(sourceDb, tmpDbPath);
  process.env.DATABASE_URL = `file:${tmpDbPath}`;
  return tmpDbPath;
}
```

Advantages:

- Tests operate against a complete, fully seeded schema without touching development or production data.
- SQLite foreign key constraints (PRAGMA foreign_keys = ON) remain fully active and enforced.
- When the test runner exits, temporary files are safely discarded by the operating system.

4. How to Write a New Test: Boilerplate Template

To add new tests to frontend/tests/:

```
import { test, describe, before, after } from "node:test";
import assert from "node:assert/strict";
import { prisma } from "../src/lib/prisma";

describe("Atelier Feature: Custom Silhouette Booking", () => {
  let createdOrderId: string;

  before(async () => {
    // Setup prerequisite test data
  });

  after(async () => {
    // Cleanup records to prevent dangling foreign keys
    if (createdOrderId) {
      await prisma.order.deleteMany({ where: { id: createdOrderId } });
    }
  });

  test("should successfully reserve fabric and record bespoke dimensions", async () => {
    const order = await prisma.order.create({
      data: {
        customerName: "Amina Khan",
        customerPhone: "+923009988776",
        shippingAddress: "Clifton Block 4",
        city: "Karachi",
        province: "Sindh",
        orderStatus: "PENDING",
        paymentStatus: "PENDING",
        paymentMethod: "COD",
        subtotalPkr: 45000,
        shippingFeePkr: 0,
        totalAmountPkr: 45000,
        tokenHash: "sample-token-hash",
      },
... [remaining code truncated for layout brevity]
```

5. CI/CD Pipeline Integration (GitHub Actions)

Include this configuration in .github/workflows/ci.yml for automated validation on every pull request:

```
name: Continuous Integration Quality Gate

on: [push, pull_request]

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js 20
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: "npm"

      - name: Install Dependencies
        run: npm ci

      - name: Verify ESLint Quality
        run: npm run frontend:lint

      - name: Verify TypeScript Compilation
        run: |
          npm run frontend:typecheck
          npm run backend:build

      - name: Execute Full Test Suite
        run: npm test
```

12. PRODUCTION DEPLOYMENT & OPERATIONS GUIDE

1. Production Architecture Options

Deploying NOIR & GOLD into production requires evaluating stateful persistence (database and media uploads) against stateless application logic.

```
flowchart TD
    DNS[Cloudflare DNS & Edge SSL] --> LB[Load Balancer / Ingress]
    LB --> NextJS[Next.js 16 Web Tier Container]
    LB --> ExpressAPI[Express 4 Catalog API Container]
    NextJS --> ManagedDB[(Managed PostgreSQL Database)]
    ExpressAPI --> ManagedDB
    NextJS --> CloudStorage[(AWS S3 / Cloudflare R2 Bucket)]
```

Option A: Fly.io Containerized Deployment (Recommended)

- Why Fly.io: Supports persistent volume mounting (/data), allowing a single low-cost virtual machine to run SQLite with automated snapshots while colocating the Next.js and Express servers.
- Reference: See docs/deployment-fly.md.

Option B: Vercel (Frontend) + Railway (Backend & Database)

- Frontend: Vercel natively hosts Next.js 16 with edge caching and automatic CI/CD git pushes.
- Backend & Database: Deploy Express and a managed PostgreSQL database on Railway or Supabase.

---

2. Multi-Stage Dockerfile Analysis

To achieve minimal container footprint (< 150 MB) and optimal security, employ a multi-stage Docker build:

```
# Stage 1: Dependency Installation
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
COPY frontend/package.json ./frontend/
COPY backend/package.json ./backend/
RUN npm ci

# Stage 2: Source Compilation
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NODE_ENV=production
RUN npm run backend:build
RUN npm run frontend:build

# Stage 3: Minimal Production Runner
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
# Run as non-root user for security hardening
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nextjs
COPY --from=builder /app/frontend/.next/standalone ./
COPY --from=builder /app/frontend/.next/static ./frontend/.next/static
COPY --from=builder /app/frontend/public ./frontend/public
USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

---

3. Database Migration Strategy: SQLite to PostgreSQL

While SQLite is lightweight and zero-maintenance for local development and initial low-traffic drops, production scale requires PostgreSQL to handle multi-threaded write concurrency and connection pooling.

Steps to Migrate:

1. Update backend/prisma/schema.prisma:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

2. Execute Prisma Migration:

```
npx prisma migrate dev --name init_postgresql
```

3. Data Transfer: Export existing records via sqlite3 dev.db .dump or utilize a Node migration script reading from SQLite and writing via Prisma to PostgreSQL.

---

4. Cloud Object Storage Migration (AWS S3 / Cloudflare R2)

In production, ephemeral containers destroy local disk uploads (/public/uploads) whenever a container restarts or scales horizontally.

Activating Cloud Storage:

Supply AWS S3 or Cloudflare R2 credentials in the production environment variables:

```
AWS_ACCESS_KEY_ID="AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
AWS_REGION="us-east-1"
AWS_S3_BUCKET_NAME="noir-and-gold-atelier-assets"
```

frontend/src/lib/storage-service.ts automatically detects these keys:

- Routes all multipart image uploads directly to S3 via @aws-sdk/client-s3.
- Returns immutable cloud CDN URLs (https://assets.noirandgold.com/...).

---

5. Production Secrets & Environment Variables

Variable Name	Environment	Description & Required Security Standard
DATABASE_URL	Production	Postgres connection URI with SSL mode enabled (sslmode=require)
ADMIN_SESSION_SECRET	Production	64-character high-entropy cryptographic passphrase
PAYMENT_WEBHOOK_SECRET	Production	Webhook secret issued by payment processor
NODE_ENV	Production	Set strictly to production (enables fail-closed guards)
PORT	Production	Bound to internal container port (e.g., 3000 or 8080)

---

6. Backup & Disaster Recovery Strategy

If operating SQLite in a single-instance container:

1. Hourly WAL Snapshots: Execute sqlite3 dev.db ".backup '/backups/backup-$(date +%s).db'" via a background cron process.
2. Offsite S3 Replication: Pipe SQLite backup snapshots immediately to an encrypted offsite cloud storage bucket.
3. Recovery RTO/RPO:

- Recovery Point Objective (RPO): Maximum 1 hour of transaction logs.
- Recovery Time Objective (RTO): Under 5 minutes to spin up a replacement container and mount the latest snapshot.

13. FULL-STACK ENGINEER LEARNING ROADMAP

1. Pedagogical Overview & Curriculum Objectives

This 12-stage curriculum is tailored for developers seeking to master modern full-stack web engineering by exploring the NOIR & GOLD codebase. Each stage provides:

- Core Concepts: Foundational principles to understand.
- Codebase References: Exact files to inspect and study.
- Hands-On Exercises: Concrete tasks to build practical mastery.

---

2. The 12-Stage Mastery Path

Stage 1: Modern JavaScript & TypeScript Fundamentals

- Core Concepts: Discriminated unions, generics, structural typing, Promises, async/await, closures, and immutability.
- Where to Study in Code:
- frontend/src/types/index.ts: Inspect discriminated unions for order and payment statuses.
- frontend/src/lib/utils.ts: Study formatting helpers, error extraction, and HMAC generators.
- Practice Exercise: Write a generic TypeScript pagination wrapper interface PaginatedResponse<T> and implement a helper function to paginate an arbitrary array of objects.

Stage 2: React 19 Fundamentals & Modern Hooks

- Core Concepts: Component composition, pure rendering, useState, useCallback, useRef, cancellation patterns in useEffect, and React 19's react-hooks/set-state-in-effect prevention.
- Where to Study in Code:
- frontend/src/components/admin/ProductsManagerView.tsx: Inspect how showFeedback and the async data loader prevent race conditions and unnecessary re-renders.
- Practice Exercise: Build a debounce hook useDebounce<T>(value: T, delayMs: number) and integrate it into a live search input field.

Stage 3: Next.js 16 App Router & Server Components

- Core Concepts: File-based routing, nested layouts, Server vs. Client component boundaries, route handlers (route.ts), and streaming SSR.
- Where to Study in Code:
- frontend/src/app/layout.tsx: Root layout and server-side metadata.
- frontend/src/app/api/v1/orders/route.ts: Handling POST requests in App Router.
- Practice Exercise: Create a new page route /lookbook featuring server-rendered high-fashion photography cards.

Stage 4: Relational Database Modeling & Prisma ORM

- Core Concepts: Relational ER diagrams, foreign key constraints, one-to-many and many-to-many relationships, soft-delete indexes, and atomic $transaction queries.
- Where to Study in Code:
- backend/prisma/schema.prisma: Analyze all 14 models and indexes.
- frontend/src/lib/order-service.ts: Study multi-model transactional checkouts.
- Practice Exercise: Add a CustomerReview model to schema.prisma with ratings (1-5) and link it to Product. Push to SQLite using npm run backend:db:push.

Stage 5: Backend REST Engineering with Express

- Core Concepts: Express routing, middleware pipelines (CORS, Morgan, JSON parsing), controller architecture, and clean error forwarding.
- Where to Study in Code:

backend/src/app.ts: Global middleware setup.

- backend/src/controllers/product.controller.ts: Request parsing and query filtering.
- Practice Exercise: Add a new Express endpoint GET /api/products/featured returning the top 3 garments ordered by creation date.

Stage 6: Authentication, Sessions & RBAC

- Core Concepts: HTTP-only cookies, SameSite policies, HMAC session signing, role-based authorization guards (ADMIN vs STAFF).
- Where to Study in Code:
- frontend/src/lib/admin-auth-guard.ts: Header extraction, cookie decryption, role validation.
- Practice Exercise: Implement a middleware check ensuring that only users with role === "SUPER_ADMIN" can trigger catastrophic data resets.

Stage 7: E-Commerce State Machines & Idempotency

- Core Concepts: Order lifecycle state transitions, payment capture states, idempotent attempt keys, and capacity-bounded refunds.
- Where to Study in Code:
- frontend/src/lib/payment-attempt-service.ts: Idempotent state machine transitions.
- frontend/src/lib/refund-ledger-service.ts: Cumulative refund capacity tracking.
- Practice Exercise: Write a unit test simulating two concurrent refund requests of PKR 50,000 against an order with an PKR 80,000 balance to verify that one fails gracefully.

Stage 8: Application Security & Hardening

- Core Concepts: OWASP Top 10 defenses, SQL injection prevention via ORMs, XSS mitigation, CSRF origin verification, and dual-layer rate limiting.
- Where to Study in Code:
- frontend/src/app/api/v1/payments/webhook/route.ts: HMAC signature verification and timestamp freshness.
- frontend/tests/security-audit.test.ts: Automated security test cases.
- Practice Exercise: Build an IP rate-limiting utility that throttles requests after 3 failed login attempts within 5 minutes.

Stage 9: Automated Testing & Test Database Isolation

- Core Concepts: Native Node test runner (node:test), assertions (assert/strict), ephemeral database cloning with mkdtempSync.
- Where to Study in Code:
- frontend/tests/order-service.test.ts: Setup, test isolation, and assertion patterns.
- frontend/tests/e2e-all-phases.test.ts: Master end-to-end integration tests.
- Practice Exercise: Write an automated test asserting that attempting to create a product variant with a negative stock count (-5) throws a validation error.

Stage 10: Performance Optimization & Data Pipelines

- Core Concepts: Database index optimization, RFC-4180 streaming CSV generation, lazy loading, and image dimension optimization.
- Where to Study in Code:
- frontend/src/app/api/v1/admin/orders/export/route.ts: Streaming CSV generation without memory spikes.
- Practice Exercise: Benchmark the CSV export route using curl with 1,000 mock orders to measure memory stability.

Stage 11: Cloud Storage & Multipart File Pipelines

- Core Concepts: HTML5 FormData, multipart HTTP decoding, file size validation, MIME sniffing, and cloud storage adapters (S3 / R2).
- Where to Study in Code:
- frontend/src/lib/storage-service.ts: Local disk vs. AWS S3 abstraction.
- frontend/src/components/admin/ImageUploader.tsx: Drag-and-drop client interface.
- Practice Exercise: Modify ImageUploader to support multi-file selection and display upload progress percentages.

Stage 12: DevOps, Docker & Architecture Design

- Core Concepts: Monorepo coordination, multi-stage Docker builds, non-root container security, environment secret injection, and

SQLite backup workflows.

- Where to Study in Code:
- Dockerfile & fly.toml: Container specifications and volume mounts.
- Practice Exercise: Build and run the production Docker container locally and access the storefront at http://localhost:3000.

14. TECHNICAL GLOSSARY & SENIOR INTERVIEW GUIDE

1. Technical Glossary (50+ Core Terms)

1. App Router: Next.js 16 file-system-based routing architecture utilizing React Server Components and nested layouts.
2. Atomic Transaction: A database execution unit ($transaction) where all operations succeed together or roll back entirely, preventing partial updates.
3. BOLA (Broken Object Level Authorization): Vulnerability where users access other users' data via direct IDs. Prevented via cryptographic HMAC tokens.
4. CORS: Cross-Origin Resource Sharing header mechanism dictating whether browser origins can access backend resources.
5. CSRF: Cross-Site Request Forgery attack mitigated in NOIR & GOLD using Sec-Fetch-Site inspection and Origin header validation.
6. CUID: Collision-resistant Unique Identifiers used for table primary keys.
7. Database Isolation: Decoupling test executions using dynamic temp databases (mkdtempSync) to prevent mutating live seed data.
8. Defense-in-Depth: Layered security architecture ensuring that the failure of any single defense layer does not compromise the system.
9. Discriminated Union: TypeScript type composed of several variants sharing a common literal property (e.g., status: "PENDING" | "PAID").
10. Dual Rate Limiter: Throttling mechanism inspecting both client IP and customer phone number to defeat bot swarms.
11. Eager Loading: Loading related entities in a single database round-trip using Prisma's include statement to eliminate N+1 queries.
12. ESLint 9 Flat Config: The updated configuration architecture for ESLint enforcing hook purity and static analysis rules.
13. Fail-Closed: Security design principle where a missing credential or unexpected failure denies access by default rather than granting it.
14. Foreign Key (FK): Column referencing the primary key of another table, maintaining referential integrity.
15. HMAC-SHA256: Hash-based Message Authentication Code verifying data integrity and authenticity using a shared cryptographic secret.
16. Hydration: React process where browser JavaScript attaches event listeners to pre-rendered HTML received from the server.
17. Idempotency: Property where performing an operation multiple times produces the exact same result as performing it once (e.g., retrying a payment attempt).
18. Immutable Audit Log: Write-only ledger table (SecurityAuditLog) that stores past operational events without edit or delete capabilities.
19. Inventory Reservation: Time-bounded hold on physical stock created during customer checkout to prevent overselling.
20. JWT (JSON Web Token): Compact URL-safe token representing signed claims between parties.
21. Lazy Loading: Deferring the loading of non-critical code or image assets until they are needed in the viewport.
22. MIME Sniffing: Verifying the actual byte headers of an uploaded file rather than relying on its file extension.
23. Monorepo: Software development strategy where multiple related projects (frontend, backend, shared configs) live in a single Git repository.
24. Morgan: Express HTTP request logging middleware used for request tracing and performance debugging.
25. Multipart Form-Data: HTTP payload format used for transmitting binary files and form fields simultaneously.
26. Node Native Test Runner: Built-in node:test execution engine eliminating third-party testing dependencies.
27. Optimistic Locking: Concurrency control method verifying that a record has not changed before applying updates (e.g., WHERE stock >= requestedQty).
28. Optimistic UI: Updating the client interface before the server response returns to deliver an instantaneous user experience.
29. Parameterized Query: Database technique separating SQL code from user-supplied data, neutralizing SQL injection vulnerabilities.
30. Prisma Schema: Declarative modeling file defining models, relations, indexes, and generator configurations.
31. Prisma Studio: Web-based GUI for visually exploring and mutating relational data.
32. RBAC: Role-Based Access Control restricting system features based on assigned roles (ADMIN vs STAFF).
33. React Server Components (RSC): React components executing strictly on the server that emit zero JavaScript to the browser

client bundle.

34. Replay Attack: Attack where a valid signed network transmission is recorded and maliciously repeated. Throttled via 300-second timestamp windows.
35. RFC-4180: Common format standard for Comma-Separated Values (CSV) files including quotation escaping rules.
36. SameSite Cookie: Security attribute preventing the browser from sending cookies along with cross-site requests.
37. SecurityAuditLog: Database table recording high-impact administrative actions (price modifications, refunds, logins).
38. Server-Side Rendering (SSR): Generating full HTML pages on the server for each request before transmitting them to the client.
39. Single Source of Truth: Designing data structures so that every piece of business data is mastered in exactly one location.
40. Soft Delete: Retaining database records while setting a timestamp flag (deletedAt = now()) to preserve historical relations.
41. State Machine: Mathematical model representing a system with distinct states and allowable transitions (e.g., order statuses).
42. Streaming Response: Transmitting HTTP response bytes progressively (via ReadableStream) to prevent memory spikes during large CSV downloads.
43. Tailwind CSS v4: Modern utility-first CSS framework with native Turbopack compilation and CSS custom properties.
44. Timing-Safe Equal: Cryptographic comparison comparing two buffers in constant time to prevent timing side-channel attacks.
45. Token Bucket: Algorithm used for rate limiting traffic burstiness.
46. Turbopack: High-performance Rust-based bundler powering Next.js development and compilation.
47. TypeScript Generic: Reusable type template enabling strong typing across diverse data structures (e.g., ApiResponse<T>).
48. useCallback: React hook that memoizes a function definition across re-renders to maintain referential equality.
49. useEffect Purity: React 19 guideline forbidding synchronous setState calls directly within effect bodies.
50. Variant Matrix: Two-dimensional grid representing garment SKUs across sizes (XS-XL) and colorways.
51. Zero-Configuration Fallback: Architectural design allowing features (e.g., image storage) to default to local disk when cloud credentials are omitted.

2. 25 Full-Stack Technical Interview Questions & Answers

1. How does NOIR & GOLD prevent inventory overselling during high-concurrency drop events?

Answer: The platform implements atomic database decrements combined with optimistic locking:

UPDATE ProductVariant SET stock = stock - :qty WHERE id = :id AND stock >= :qty.

If another concurrent transaction has already purchased the remaining stock, the condition fails, zero rows are affected, and the application throws INSUFFICIENT_STOCK. Additionally, an InventoryReservation row with a 15-minute expiration window holds the stock during checkout.

2. Why was SQLite selected for this project, and how would you transition to PostgreSQL in production?

Answer: SQLite requires zero configuration, stores data in a single file (dev.db), and delivers microsecond in-process reads, making it ideal for prototyping, testing, and single-instance deployments. To transition to PostgreSQL, we update the datasource db provider in backend/prisma/schema.prisma from sqlite to postgresql, generate a migration with prisma migrate dev, and configure a managed connection pool (e.g., PgBouncer) in production.

3. What is the difference between React Server Components (RSC) and Client Components in Next.js 16?

Answer: Server Components run exclusively on the server at request time, have direct access to backend resources (databases, file systems), and send pure HTML to the client with zero JavaScript footprint. Client Components (declared with "use client") are bundled and hydrated in the browser, enabling React hooks (useState, useEffect), event listeners, and interactive UI logic.

4. Explain the soft-delete architecture implemented in this project and why physical deletes are prohibited.

Answer: Physical row deletion (DELETE FROM Product) breaks relational foreign keys and destroys historical invoices and receipts. We add a nullable deletedAt DateTime? column with an index @@index([deletedAt]). Active queries filter for where: { deletedAt: null }. Archiving a garment sets deletedAt = new Date(), which removes it from the catalog while preserving historical order records.

5. How are payment webhooks protected against forgery and replay attacks?

Answer: Every incoming webhook must supply x-payment-signature and x-payment-timestamp. The server computes an HMAC-SHA256 hash using the server secret PAYMENT_WEBHOOK_SECRET over the raw payload and verifies it using crypto.timingSafeEqual. To block replay attacks, the server verifies that the timestamp is within 300 seconds of the server's current time.

6. Why did we eliminate the DEFAULT_WEBHOOK_SECRET fallback?

Answer: Hardcoded fallback secrets in source code allow attackers to forge webhook payloads if environment secrets are not configured. Eliminating the fallback enforces a "fail-closed" security policy: if PAYMENT_WEBHOOK_SECRET is unset in production, the server immediately returns HTTP 500 rather than accepting forged signatures.

7. How does the admin authentication system enforce Role-Based Access Control (RBAC)?

Answer: When an admin logs in, an HMAC-signed session cookie (admin-token) is issued encoding their email and role (ADMIN or STAFF). The admin-auth-guard.ts middleware verifies this cookie on every protected request. Privileged operations (modifying prices, issuing refunds, archiving products) require requireAdminRole(), rejecting staff with HTTP 403 Forbidden.

8. Explain how the capacity ledger prevents refund overpayment.

Answer: The refund service performs an aggregate query: $\sum \text{Refunds where status = COMPLETED}$. The maximum refundable capacity is calculated as $\text{Captured Total} - \text{Total Prior Refunds}$. If an admin attempts to refund more than this capacity, the service throws CAPACITY_EXCEEDED and aborts the transaction.

9. Why does the application use streaming for CSV export rather than generating the file in memory?

Answer: Accumulating thousands of order records into an in-memory string can cause memory exhaustion and crash the Node process. The export route (/api/v1/admin/orders/export) creates a ReadableStream that transforms and pipes database chunks directly to the HTTP client using RFC-4180 escaping, keeping server RAM usage constant regardless of dataset size.

10. How are test suites isolated to prevent corrupting the development database?

Answer: Tests use fs.mkdtempSync to create an ephemeral temporary directory and copy backend/prisma/dev.db into it. The DATABASE_URL is dynamically pointed to this copy. All writes occur against the isolated clone, keeping development data pristine.

11. What is the purpose of Sec-Fetch-Site inspection in the CSRF guard?

Answer: Sec-Fetch-Site is an HTTP request header sent by modern browsers indicating the relationship between the origin making the request and the target origin. If Sec-Fetch-Site: cross-site, the request originated from a foreign domain, allowing the server to block Cross-Site Request Forgery (CSRF) attempts immediately.

12. How does the image upload pipeline support both local storage and cloud S3?

Answer: The storage service (storage-service.ts) inspects environment variables. If AWS_ACCESS_KEY_ID and AWS_S3_BUCKET_NAME are configured, it uses @aws-sdk/client-s3 to upload directly to S3. If unconfigured, it seamlessly falls back to saving files in public/uploads/, providing a zero-configuration developer experience.

13. How does React 19 handle lifecycle state updates in useEffect?

Answer: React 19's ESLint rules strictly flag synchronous state mutations directly inside useEffect (react-hooks/set-state-in-effect) because they trigger cascading re-renders and race conditions. We resolve this by declaring an asynchronous loader with a cancellation flag (let ignore = false; return () => { ignore = true; }) or driving state through event handlers.

14. What is the role of tokenHash in the Order model?

Answer: In guest checkout, customers do not have user accounts or passwords. To allow them to view order tracking (/orders/[id]?token=...) without exposing orders to enumeration attacks (BOLA / IDOR), the server issues a cryptographically random HMAC token and stores its SHA-256 hash in tokenHash. Requests must present the matching token to view the order.

15. Explain how courier dispatching is validated in the order state machine.

Answer: Transitioning an order from PROCESSING to SHIPPED enforces business invariants: the update payload must include both courierName (e.g., TCS, Leopards) and trackingNumber. If either field is missing, the service rejects the transition, ensuring customers are never given a "Shipped" status without real tracking information.

16. How does the dual rate limiter differentiate between malicious bots and legitimate users?

Answer: It checks two dimensions independently: the client's IP address (max 10 orders per 15 min) and the customer's phone

number (max 5 orders per 15 min). This blocks distributed botnets that rotate IP addresses while using the same phone number, as well as single-source automated scrapers.

17. How does the application prevent SQL injection vulnerabilities?

Answer: All database access is routed through Prisma ORM. Prisma parses queries into parameterized SQL statements where parameters are passed separately to the database engine rather than concatenated as strings, neutralizing SQL injection vectors.

18. Why is the VariantMatrixBuilder crucial for luxury couture e-commerce?

Answer: Haute couture garments are produced in bespoke combinations of sizes (XS to Custom) and artisanal fabric colorways. The VariantMatrixBuilder enables administrators to dynamically generate SKUs, assign individual stock levels, configure price deltas, and preview color swatches within a single unified administrative view.

19. What is the significance of the SecurityAuditLog table?

Answer: High-risk actions—such as administrative logins, price modifications, product archiving, and refund issuance—must have an immutable audit trail. SecurityAuditLog records who performed the action, target resources, client IP addresses, timestamps, and JSON snapshots of state changes for auditing and dispute resolution.

20. How does the frontend handle responsive image loading and cumulative layout shift (CLS)?

Answer: Next.js <Image /> requires explicit aspect ratios (width and height or fill). It automatically generates responsive srcset attributes and serves modern formats (WebP/AVIF). This reserves space in the DOM before image assets finish downloading, maintaining a Cumulative Layout Shift (CLS) of 0.

21. How do Express controllers differ from Next.js route handlers?

Answer: Express controllers receive (req: Request, res: Response) objects and rely on Express middleware (CORS, Morgan, body-parser) for request processing. Next.js App Router route handlers export named HTTP functions (GET, POST) receiving standard Web API Request objects and returning Web API Response / NextResponse instances.

22. What happens to inventory when an order is cancelled?

Answer: The order cancellation service executes an atomic transaction that: (1) transitions orderStatus to CANCELLED, (2) restores the exact quantities of each item back to their respective ProductVariant.stock, (3) marks associated InventoryReservation rows as released, and (4) logs the event in InventoryAuditLog.

23. What are the benefits of using an npm workspace monorepo?

Answer: Monorepos enable sharing schemas and database models across frontend and backend packages without duplicating code or publishing private npm packages. They allow running synchronized test suites across all layers and keeping configuration files unified.

24. How does the application ensure timing-attack resistance during HMAC verification?

Answer: Comparing string hashes using standard equality (hashA === hashB) returns false as soon as the first non-matching byte is found, leaking timing information to attackers. We use crypto.timingSafeEqual(), which compares buffers in constant time regardless of where mismatches occur.

25. How is the Next.js production build optimized?

Answer: Next.js analyzes the component tree during next build, performs tree-shaking to eliminate unused code, generates static HTML for static pages, compiles CSS with Tailwind v4, and generates optimized server-side bundles with serverExternalPackages configured to prevent duplicating heavy native modules.